Exercice ETL — Givre & Co

Contexte

Givre & Co est une petite chaîne de glaciers belge (7 magasins). La direction se demande :

> *« Est-ce qu'on vend vraiment plus quand il fait chaud ? Et est-ce que ça vaut la peine d'ouvrir à la côte en hiver ? »*

Pour y répondre, vous devez alimenter un entrepôt de données à partir de :

1. la **base applicative** `GlacierApp` (caisse des magasins, année 2025) ;
2. l'API **Nominatim** (OpenStreetMap) pour géolocaliser les magasins ;
3. l'API **Open-Meteo** (archives) pour récupérer la météo réelle de chaque jour à chaque magasin ;
4. l'API **OpenAI** pour compléter les informations manquantes sur les produits.

Mise en place

1. Exécuter `01_source_GlacierApp.sql` → crée et remplit la base source.
2. Exécuter `02_dw_GlacierDW.sql` → crée l'entrepôt ; `dim_date` est déjà remplie.
3. Réaliser l'ETL en **Python** (`httpx`, `sqlalchemy`, `pyodbc`).

Source : `GlacierApp`

Table	Contenu
`categorie`	catégories de produits
`produit`	catalogue (prix **actuel**)
`magasin`	adresses des magasins (sans coordonnées)
`client`	porteurs de carte de fidélité
`commande`	entête de ticket (date/heure, magasin, statut…)
`ligne_commande`	détail : produit, quantité, prix **au moment de la vente**, remise en %

Cible : `GlacierDW`



`dim_produit`

- `categorie` : nom de la catégorie ; si elle est absente dans la source, la demander à **OpenAI** (parmi les catégories existantes uniquement).

- `type_produit` : `'Chaud'` pour *Gaufres & crêpes* et *Boissons chaudes*, sinon `'Froid'`.
- `allergenes` et `est_vegan` : n'existent pas dans la source → à obtenir via **OpenAI**.
- Les noms doivent être propres (pas d'espaces parasites, majuscule initiale).

`dim_magasin`

- `ville` doit être nettoyée et homogène (ex. `Liège`, `Knokke-Heist`).
- `latitude`, `longitude`, `region` viennent de **Nominatim**.
- `est_cotier` = 1 si la longitude est inférieure à **3.35**.
- Le magasin de test (inactif) ne doit **pas** se retrouver dans l'entrepôt.

`fait_vente`

- Grain : **1 ligne = 1 produit × 1 magasin × 1 jour**.
- Seules les commandes **payées** sont prises en compte.
- `montant_brut = Σ quantite × prix_unitaire` (prix de la ligne de commande !)
- `montant_remise = Σ quantite × prix_unitaire × remise_pct / 100`
- `montant_net = montant_brut − montant_remise`
- `nb_commandes` = nombre de commandes distinctes contenant ce produit ce jour-là.
- `temp_max`, `temp_min`, `precipitation_mm` viennent d'**Open-Meteo** pour le magasin et le jour.

Les API

Nominatim — géocodage

```
GET https://nominatim.openstreetmap.org/search
    ?street=Veldstraat 50&city=Gent&postalcode=9000&country=Belgium
    &format=jsonv2&addressdetails=1&limit=1
```

- Réponse : une **liste** ; on utilise `lat`, `lon` (des chaînes !) et `address.state`.
- ⚠️ **Règles d'utilisation** : un en-tête `User-Agent` identifiant votre application est **obligatoire**, et **1 requête par seconde maximum**. Ici, pas de `asyncio.gather` : on fait les appels un par un avec une pause.
- Si une adresse n'est pas trouvée, prévoir un plan B (par ex. rechercher uniquement la ville).

Open-Meteo — météo historique (sans clé)

```
GET https://archive-api.open-meteo.com/v1/archive
    ?latitude=51.05&longitude=3.72
    &start_date=2025-01-01&end_date=2025-12-31
    &daily=temperature_2m_max,temperature_2m_min,precipitation_sum
    &timezone=Europe/Brussels
```

- Un seul appel par magasin donne toute l'année.
- Réponse : `data["daily"]["time"]`, `data["daily"]["temperature_2m_max"]`, … (listes alignées).
- Ici les appels sont indépendants → idéal pour `httpx.AsyncClient` + `asyncio.gather`.

OpenAI — compléter les données produits

```
import json
from httpx import AsyncClient

API_KEY = "..."   # ne jamais committer la clé ! (variable d'environnement,
fichier .env)

prompt = (
    "Tu es un assistant pour un glacier belge. Pour le produit 'Cornet spéculoos',
"
    "réponds uniquement en JSON avec les clés : "
    "categorie (une valeur parmi : Glaces, Sorbets, Gaufres & crêpes, Boissons
chaudes, "
    "Boissons fraîches, Milkshakes), allergenes (liste de chaînes), est_vegan
(booléen)."
)

async with AsyncClient(timeout=30) as client:
    response = await client.post(
        "https://api.openai.com/v1/chat/completions",
        headers={"Authorization": f"Bearer {API_KEY}"},
        json={
            "model": "gpt-4o-mini",                      # ou le modèle indiqué
en cours
            "messages": [{"role": "user", "content": prompt}],
            "response_format": {"type": "json_object"},   # force une réponse JSON
            "temperature": 0,
        },
    )
    data = json.loads(response.json()["choices"][0]["message"]["content"])
```

- Un appel par produit (19 produits) → `asyncio.gather` convient.
- **Ne jamais faire confiance aveuglément** à la réponse : vérifier que `categorie` fait bien partie de la liste, que `est_vegan` est un booléen, etc. Sinon, valeur par défaut (`'Non classé'`, `NULL`).
- Les réponses d'un modèle peuvent varier d'une exécution à l'autre : c'est une raison de plus pour les mettre en cache.

Étapes attendues

1. **Extract** : lire les tables utiles de `GlacierApp` ; appeler les deux API.
2. **Transform** : nettoyer, filtrer, agréger au bon grain, rattacher la météo.
3. **Load** : remplir `dim_produit` et `dim_magasin`, puis récupérer leurs clés de substitution pour charger `fait_vente`.
4. Le script doit pouvoir être **relancé** sans créer de doublons.

Questions d'analyse (sur le DW)

1. Chiffre d'affaires net par magasin et par mois.
2. Ventes moyennes par jour selon des tranches de `temp_max` (< 10 °C, 10–20 °C, 20–25 °C, > 25 °C).

3. Part des produits *Chauds* / *Froids* selon la saison.
4. Les jours de pluie font-ils baisser le chiffre d'affaires ?
5. Les magasins côtiers sont-ils plus sensibles à la météo que les autres ?

Bonus

- Mettre en cache les réponses des API dans des fichiers JSON pour ne pas les rappeler à chaque exécution.
- Ajouter une table `etl_log` (date d'exécution, nombre de lignes chargées, durée).